

RESEARCH PROJECT BLUEPRINT

When Imbalance Correction Backfires

Robustness of Tabular Imbalance Mitigation Under Minority-Label Contamination

Three-day empirical ML study and manuscript-development plan

Prepared: August 12, 2026

Version: Final updated research blueprint (literature verified through August 12, 2026)

Status: Experimental protocol - no empirical results have been generated yet

RESEARCH INTEGRITY RULE

All reported numerical results must come from completed experiments. The document pre-registers hypotheses and analysis logic; it does not contain fabricated findings. Any model, dataset, or run dropped for technical reasons must be documented before inspecting comparative performance.

Document Purpose

This document updates and extends the earlier research blueprint; it does not discard the established project structure. The core direction remains whether methods intended to correct class imbalance remain beneficial when rare-class supervision is unreliable. The final protocol now separates two directed label-error mechanisms: (A) majority-to-minority contamination, where majority examples are mislabeled as minority, and (B) minority-to-majority label loss, where true minority examples are mislabeled as majority. All previously useful components - controlled imbalance, clean held-out testing, TabPFN/GBDT comparison, calibration analysis, reproducibility rules, figures, statistics, and the three-day plan - are retained.

UPDATE PRINCIPLE. This final file is cumulative. New evidence is added to the prior plan, earlier valid methods are preserved, and only items found to be inaccurate, outdated, or unsafe for inference are corrected explicitly.

At-a-Glance Decision

Item	Frozen decision
Core question	When rare-class supervision is unreliable, when do imbalance corrections help, stop helping, or become harmful - and does the answer differ for majority->minority contamination versus minority->majority label loss?
Primary model family	TabPFN / prior-data fitted tabular foundation models.
Primary correction methods	TabPFN threshold correction, downsampling, DistPFN, DistPFN-T; XGBoost weighting, random oversampling, and SMOTE.
Noise mechanism	Two directed mechanisms are core: (A) majority -> minority contamination, where false rare labels enter the observed minority pool; and (B) minority -> majority label loss, where true rare labels are omitted. The $c=0$ clean condition is shared.
Imbalance levels	True minority prevalence $r = 0.50, 0.20, 0.10, 0.05$ in the clean training pool.
Directed noise levels	$c = 0, 0.05, 0.10, 0.20, 0.30$ within each direction; $c=0$ is run once as the shared clean condition. Direction A and B use the direction-specific definitions in Section 6.
Datasets	Five public binary UCI datasets; fixed clean test set for every condition.
Primary outcomes	Balanced accuracy, worst-class accuracy/recall, AUPRC, macro-F1, correction gain, correction harm rate, correction break-even point.
Reliability outcomes	Brier score, log loss, expected calibration error (ECE).
Estimated workload	540 unique dataset-seed-stress contexts; roughly 3,240 expensive fit/inference jobs and 4,860 evaluated method-condition rows because several TabPFN corrections are post-hoc.

Recommended paper title

When Imbalance Correction Backfires: Robustness of Tabular Imbalance Mitigation Under Minority-Label Contamination

Novelty wording to use

We do NOT claim that class imbalance, label noise, or their combination is new. We study a narrower mechanism: the robustness of modern imbalance-correction strategies under two directed rare-label errors - false majority examples entering the minority-labeled pool and true minority examples being lost to the majority label - including the direction-specific point at which a correction becomes worse than applying no correction.

Contents

- 1. Research Motivation and Post-CLIMB Gap
- 2. Literature Basis and What Came After CLIMB
- 3. Research Questions and Hypotheses
- 4. Datasets
- 5. Controlled Experimental Design
- 6. Directed Minority-Label Error Protocol
- 7. Correction Strategies and Models
- 8. Preprocessing and Data Leakage Controls
- 9. Evaluation Metrics
- 10. New Contribution Metrics: Gain, Harm, Break-Even, Prior Sensitivity
- 11. Statistical Analysis
- 12. Required Figures and Tables
- 13. Implementation Blueprint
- 14. Three-Day Execution Schedule
- 15. Paper Structure
- 16. Threats to Validity
- 17. Reproducibility and Run-Logging Rules
- 18. Decision Logic After Results
- 19. References
- Appendix A. Immediate Start Checklist
- Appendix B. Manuscript Finalization Checklist

1. Research Motivation and Post-CLIMB Gap

Class-imbalanced tabular learning is mature, and recent benchmarks already make a broad "imbalance benchmark" difficult to justify. CLIMB evaluates 29 class-imbalanced learning algorithms on 73 tabular datasets. TILBench, released in 2026, expands systematic imbalanced-learning evaluation to more than 40 methods across 57 datasets and over 200,000 controlled experiments. A new paper that simply compares classifiers or resampling methods under imbalance would therefore be weakly differentiated [1], [2].

The post-CLIMB literature also closes several nearby gaps. McDowell et al. show that TabPFN suffers on rare classes and that simple thresholding (τ equal to the minority training prior) and downsampling can correct much of this bias [3]. Lee introduces DistPFN and DistPFN-T, which adjust TabPFN probabilities using the observed training prior and predicted posterior under label shift [4]. Hu and Ghelichi report notable TabPFN resilience to controlled label noise [5]. ITBoost specifically targets label-noise robustness in gradient boosting [6]. SkillTFM, submitted August 6, 2026, evaluates training-free TFM adaptation across multiple boundary families including imbalance, label noise, and mixed-boundary settings [8].

The defensible opportunity is therefore not "Does imbalance plus noise hurt?" The sharper question is whether the correction mechanism itself becomes unreliable when rare-class supervision is untrustworthy. We test two mechanisms separately because they can damage correction in different ways. Majority-to-minority contamination inserts false rare examples that weighting, oversampling, SMOTE, and minority-favoring thresholds may amplify; minority-to-majority label loss removes genuine rare examples and deflates the observed minority prior, which can distort prior-dependent correction and minority recall. The directions are analyzed separately rather than collapsed into one generic noise condition.

Central mechanism

Imbalance correction increases minority influence or uses the minority prior. Direction A can inject false minority-labeled examples that corrections may amplify; Direction B can remove genuine minority examples and deflate the observed prior. The paper measures, separately for each direction, when correction benefit collapses and when correction becomes harmful.

2. Literature Basis and What Came After CLIMB

Work	Date / venue	What it establishes	How this study differs
CLIMB [1]	NeurIPS 2025	Large benchmark of tabular class-imbalanced learning; includes robustness perturbations.	Starting point, not the frontier. We focus on correction failure under contaminated rare labels.
TILBench [2]	May 2026	40+ methods, 57 datasets, >200k experiments across imbalance regimes.	Confirms that another generic imbalance leaderboard is unnecessary.
McDowell et al. [3]	May 20, 2026 / arXiv preprint	For PFNs, threshold τ = minority prior and downsampling are strong imbalance corrections.	We stress-test those corrections when the observed minority labels/prior are corrupted.
DistPFN / DistPFN-T [4]	ICML 2026 / arXiv v2 May 24, 2026	Test-time posterior adjustment for TabPFN-family models; evaluates 253 OpenML datasets and uses the training prior in the adjustment.	We test whether prior-dependent adjustment is robust when training labels distort that prior.
Noise Immunity [5]	Apr. 2026	TabPFN remains strong under several synthetic perturbations including label noise.	We use structured minority-label contamination and evaluate correction mechanisms, not only base-model robustness.
ITBoost [6]	May 2026	Robust boosting that downweights samples with noisy residual trajectories.	Useful context/optional baseline for a label-noise-aware classical method.
Uncertainty benchmark [7]	ESANN 2026	TFMs can have strong AUC while uncertainty reliability remains weaker.	Motivates Brier/log-loss/ECE alongside discrimination metrics.

Work	Date / venue	What it establishes	How this study differs
SkillTFM [8]	Aug. 6, 2026 / arXiv preprint	Training-free adaptation of TFMs across simulated boundary settings and a real-world forecasting task; broader than a single imbalance-correction mechanism.	Our study is mechanism-specific: whether a named imbalance correction becomes harmful when rare labels are contaminated.
Tabular Numeric Stretch Transformation [22]	Aug. 10, 2026 / arXiv preprint	Recent tabular-learning work; Appendix G injects symmetric label noise only into training labels while keeping validation/test sets clean.	It is not an imbalance-correction study, but it independently supports our clean evaluation-set design and shows that label-noise robustness remains an active 2026 topic.
Robust-GBDT [23]	2023-2024 / arXiv preprint	Explicitly studies GBDT classification under label noise and class imbalance using robust/nonconvex losses.	Prevents an overclaim that imbalance + label noise is new; our narrower contribution concerns correction failure in modern TFM/PFN imbalance mitigation.
GBDT label-noise study [24]	2024-2025 / arXiv preprint	Studies GBDTs on tabular data with controlled label noise and noise-detection/correction methods.	Supports the need for a classical noise-robust context but does not test TFM imbalance corrections under contaminated rare labels.
Foundation-model-prior active learning [25]	May 30, 2026 / arXiv preprint	Studies active learning under class imbalance and noisy labels across image/text settings.	Adjacent evidence that the dual challenge is not new; it is outside tabular imbalance-correction stress testing and therefore not a competing implementation.

2.1 What the paper can legitimately claim

- A controlled robustness study of correction strategies, not a new imbalance algorithm.
- A mechanism-focused evaluation of two directed rare-label error processes - majority-to-minority contamination and minority-to-majority label loss - with identical clean test sets and constant clean training size across imbalance ratios.
- A practical correction break-even point: the contamination level beyond which a correction is no better than no correction.
- A prior-sensitivity diagnostic that compares observed-prior correction with an oracle clean-prior variant, helping separate prior corruption from model degradation.
- A reliability analysis that measures whether a correction can preserve ranking performance while degrading probability quality.

2.2 Literature Verification Protocol and Cutoff

Cutoff date. The literature review in this blueprint was re-checked on August 12, 2026. The manuscript must describe this as a search cutoff, not as proof that no competing work exists.

- Primary-source priority: use peer-reviewed proceedings, OpenReview, official arXiv records, UCI dataset pages, and authors' official repositories when implementation details are needed.
- Core search concepts: tabular foundation models, TabPFN, prior-data fitted networks, class imbalance, label shift, minority class, asymmetric or targeted label noise, noisy labels, imbalance correction, calibration, and robust boosting.
- Novelty discipline: do not claim that class imbalance plus label noise is new. Prior work already studies that broader combination, including robust GBDT methods.
- Current gap framing: test whether imbalance-correction mechanisms remain beneficial under two distinct directed rare-label errors - false majority examples entering the minority-labeled pool and true minority examples being lost to the majority label - and identify the direction-specific noise level at which correction ceases to help.

- Preprint discipline: papers available only as preprints must be cited as preprints unless a primary venue page verifies publication status.
- Submission refresh: immediately before submission, repeat the search and revise the novelty paragraph if a newer overlapping paper has appeared.

2.3 Exact Novelty Wording to Use

Recommended wording: “Recent work has separately advanced imbalance correction for tabular foundation models and studied robustness to label noise. We therefore focus on a narrower question: whether imbalance-correction mechanisms remain beneficial under two directed minority-label error processes - majority-to-minority contamination and minority-to-majority label loss - and when those corrections cross from helpful to harmful.”

Avoid absolute phrases such as “first study,” “no previous work,” or “latest method” unless a documented, reproducible literature search at submission time supports them. The contribution should be stated as a controlled mechanism-focused stress test, not as a claim that noisy imbalanced learning itself is new.

3. Research Questions and Hypotheses

3.1 Research Questions

RQ1. How do majority-to-minority contamination and minority-to-majority label loss separately change the benefit of imbalance correction as the true minority class becomes rarer?

RQ2. For each label-error direction, at what noise level does each correction strategy reach a break-even point where it no longer improves over the corresponding uncorrected model?

RQ3. Are prior-dependent TabPFN corrections (thresholding, DistPFN, DistPFN-T) differently sensitive when the observed minority prior is inflated by majority-to-minority contamination versus deflated by minority-to-majority label loss?

RQ4. Does correction under either directed rare-label error damage probability reliability (Brier score, log loss, ECE) before it visibly damages ranking/discrimination performance?

RQ5 (diagnostic). How much of correction failure in each direction is attributable specifically to using a distorted observed prior, as measured by an oracle clean-prior ablation?

3.2 Pre-registered Hypotheses

- H1. The gain from aggressive minority amplification (random oversampling, SMOTE, and class weighting) will shrink most clearly under majority-to-minority contamination, because false minority-labeled examples can be upweighted, duplicated, or used to synthesize additional minority samples; the effect is expected to be strongest under severe true imbalance.
- H2. TabPFN threshold correction will perform well when labels are clean, consistent with recent PFN imbalance results, but its behavior will differ by direction because majority-to-minority contamination inflates the observed minority prior whereas minority-to-majority label loss deflates it.
- H3. DistPFN and DistPFN-T will be sensitive to prior distortion because their adjustment explicitly uses the training prior; observed-prior versus oracle-prior gaps will reveal whether inflation and deflation of the minority prior affect the corrections differently.
- H4. Downsampling will be less likely than oversampling to amplify false minority-labeled points under majority-to-minority contamination, but minority-to-majority label loss may instead remove useful rare examples before correction; both tradeoffs are expected to be dataset-dependent.
- H5. Calibration/reliability metrics will often reveal correction harm earlier than AUROC.

Important

These are hypotheses, not expected findings. The paper must report contradictory results if the experiments do not support them.

4. Datasets

The core study keeps the same five public binary UCI datasets because they are small enough for rapid repeated TabPFN evaluation, span multiple domains, and provide enough examples to construct controlled imbalance conditions. Exact natural class counts must be recorded by the loader before any resampling.

Dataset	UCI ID	N	Features	Feature type	Domain / target
Banknote Authentication	267	1,372	4	Numeric	Counterfeit vs genuine banknote
Blood Transfusion Service Center	176	748	4	Numeric	Whether a donor donated in the target period
South German Credit	522	1,000	20 predictors	Mixed / encoded categorical + numeric	Credit risk: good vs bad
Spambase	94	4,601	57	Numeric	Spam vs non-spam email
Breast Cancer Wisconsin (Diagnostic)	17	569	30	Numeric	Malignant vs benign diagnosis

4.1 Dataset rules

- Define one semantically meaningful positive/minority class per dataset and keep it fixed across all runs.
- Drop identifier-only fields; do not use test labels in any preprocessing, correction selection, threshold tuning, or hyperparameter tuning.
- Create one stratified 80/20 clean train/test split for each seed before controlled imbalance or contamination is introduced.
- Keep each seed's test rows and true test labels unchanged across every imbalance, directed label-error, and correction condition.
- If categorical variables are one-hot encoded, fit the encoder on training data only and apply it to the fixed test set.

4.2 Dataset Audit and Final Dataset Choice

- Banknote Authentication (UCI 267): 1,372 instances and 4 predictors. Use the UCI target as provided; verify class counts after loading rather than assuming which encoded label is the minority.
- Blood Transfusion Service Center (UCI 176): 748 instances and 4 predictors. Verify the target encoding and class counts directly after download.
- South German Credit (UCI 522): 1,000 instances and 20 predictors. Use this corrected representation instead of the legacy Statlog German Credit dataset because UCI explicitly warns that the older representation contains severe coding/documentation problems.
- Spambase (UCI 94): 4,601 instances and 57 predictors. Because the current UCI metadata is not fully consistent about missingness, audit missing values programmatically and record the observed counts.
- Wisconsin Diagnostic Breast Cancer (UCI 17): 569 instances and 30 predictors. Remove the identifier column from predictors and use diagnosis as the target.
- For every dataset, save the raw download, retrieval date, UCI ID, DOI, target column, feature list, raw class counts, missing-value counts, and any transformations in a machine-readable dataset manifest. Do not infer undocumented semantics.

5. Controlled Experimental Design

The core experiment uses true clean minority prevalence together with a directed label-error factor. For each nonzero noise level, the same clean controlled training set produces two separate noisy versions: (A) majority-to-minority contamination and (B) minority-to-majority label loss. Corrections are applied after the directed

label error, exactly as a practitioner would see the data. The clean condition is shared once, and the test set remains unchanged and clean.



Analyze directions separately: correction gain, harm rate, break-even noise, calibration, and prior sensitivity

Figure 1. Controlled pipeline for separately testing majority-to-minority contamination and minority-to-majority label loss before applying the same imbalance-correction strategies.

5.1 Factorial grid

Factor	Levels	Purpose
Label-error direction d	A: majority -> minority; B: minority -> majority; c=0 shared clean	Separates false-minority amplification from true-minority omission/prior deflation.
True minority prevalence r	0.50, 0.20, 0.10, 0.05	Balanced through severe true training imbalance before corruption.
Directed label-error rate c	0, 0.05, 0.10, 0.20, 0.30	Direction-specific rate: observed-minority contamination for A; minority-label loss for B.
Random seeds	13, 37, 73	Controls split, clean subsampling, and contamination sampling.
Datasets	5	Cross-domain replication within three-day compute budget.
Correction pipelines	9 core + oracle diagnostics	Compares uncorrected, prior-based, downsampling, weighting, oversampling, and SMOTE families.

5.2 Constant clean training size across imbalance levels

To avoid confounding imbalance with total training-set size, use one common clean training size N_{common} for all r values within a dataset/seed. Compute the maximum feasible N for each target r from the available clean majority and minority pools, then choose the minimum across r . Sample exactly N_{common} rows at every r .

```

feasible_N(r) = min(n_min_available / r,
                   n_maj_available / (1-r))
N_common = floor(min_r feasible_N(r))

for r in [0.50, 0.20, 0.10, 0.05]:
    n_min = round(r * N_common)
    n_maj = N_common - n_min
    sample without replacement from the clean training split
  
```

5.3 Natural-reference condition

For context only, also fit the uncorrected baseline models on the full clean 80% training split with its natural class prevalence. Keep this reference out of the factorial hypothesis tests; it exists to show how far the controlled conditions are from the original dataset.

6. Directed Minority-Label Error Protocol

6.1 Direction A - majority examples mislabeled as minority

Direction A models contamination of the observed rare-class set. Starting from a clean controlled training set with $N_{\min}$ true minority examples, choose a target contamination fraction c : the desired fraction of observed minority-labeled examples that are actually majority. Flip k true-majority labels from 0 to 1 while keeping all feature values unchanged.

```
Direction A: majority -> minority contamination
# Desired:  $k_A / (N_{\min} + k_A) = c$ 
 $k_A = \text{round}((c / (1 - c)) * N_{\min})$ 
 $\text{idx}_A = \text{random\_sample}(\text{true\_majority\_indices}, k_A)$ 
 $y_A[\text{idx}_A] = 1$ 
 $\text{realized}_A = k_A / (N_{\min} + k_A)$ 
```

```
Direction B: minority -> majority label loss
# Desired:  $k_B / N_{\min} = c$ 
 $k_B = \text{round}(c * N_{\min})$ 
 $\text{idx}_B = \text{random\_sample}(\text{true\_minority\_indices}, k_B)$ 
 $y_B[\text{idx}_B] = 0$ 
 $\text{realized}_B = k_B / N_{\min}$ 
```

For $c=0$, store one shared clean condition. Feature values never change.

This direction is intentional: random oversampling, SMOTE, and class weighting treat the mislabeled rows as rare examples and may therefore increase their influence. Threshold and DistPFN-family methods also observe an inflated minority prior. The clean test labels preserve ground truth, so any apparent benefit or harm is measurable.

6.2 Direction B - minority examples mislabeled as majority (rare-class label loss)

Direction B is part of the core matrix, not an optional follow-up. Starting from the same clean controlled training set, choose a label-loss rate c and flip $k = \text{round}(c \times N_{\min})$ true minority labels from 1 to 0. Here c is the fraction of true minority labels that are omitted. This creates the complementary failure mode in which genuine rare events are missed during annotation and the observed minority prior becomes too small.

6.3 Why the two directions are analyzed separately

The numeric rate c is direction-specific and should not be treated as the same physical corruption process across directions. In Direction A, c is the fraction of the observed minority-labeled pool that is actually majority; in Direction B, c is the fraction of true minority labels that are lost. Therefore, primary break-even and harm analyses are computed within each direction. Cross-direction interpretation should use the number of flipped labels, the change in observed class prior, and performance effects rather than assuming that equal c values imply identical supervision damage.

6.4 Validation checks for every corrupted dataset

- Assert that feature rows are unchanged; only labels may differ from the clean controlled training set.
- Assert that the test set is byte-for-byte identical across contamination levels within a dataset/seed.
- Record noise_direction, the true clean prior, observed noisy prior, requested direction-specific rate c , realized rate, and number of flipped rows. For Direction A also record the realized fraction of the observed minority-labeled set that is truly majority; for Direction B record the realized fraction of true minority labels omitted.
- At $c = 0$, run one shared clean condition: noisy labels must exactly equal the clean labels and no direction-specific duplicate is needed.
- Never regenerate directed label errors separately for competing correction methods; all methods receive the identical noisy labels for a dataset/seed/r/c/direction cell.

7. Correction Strategies and Models

The study compares correction strategies rather than a large zoo of classifiers. TabPFN corrections are prioritized because 2026 work proposes explicit PFN-specific imbalance and label-shift corrections. XGBoost provides a strong classical control with common imbalance remedies.

Pipeline	Implementation	Why included
TabPFN - none	TabPFNCClassifier; default decision threshold 0.5; record exact package/checkpoint.	Uncorrected TFM reference.
TabPFN - threshold	Use TabPFN probabilities; classify minority if $p(y=1) > \tau_{\text{obs}}$, where τ_{obs} = observed noisy minority prior.	Directly stress-tests the threshold rule motivated by McDowell et al. [3].
TabPFN - downsample	Downsample observed majority labels until observed classes are 1:1, then run TabPFN.	Strong PFN data-level correction reported in [3].
DistPFN	Use the verified paper formula: $p_{\text{tilde}}(y) = \text{Norm}[p_{\text{hat}}(y)^2 / p_{\text{train}}(y)]$. For the main observed-prior variant use the noisy training-label prior; log every class prior explicitly.	Prior-dependent TFM correction from [4].
DistPFN-T	Use the verified paper formula: $\tau = \text{CE}(p_{\text{hat}}, p_{\text{train}})$; $p_{\text{hat_T}} = \text{softmax}(p_{\text{hat}}/\tau)$; $p_{\text{tilde}}(y) = \text{Norm}[p_{\text{hat}}(y) * p_{\text{hat_T}}(y)/p_{\text{train}}(y)]$. Validate against the authors' official implementation before the full matrix.	Adaptive prior-dependent correction.
XGBoost - none	Frozen XGBoost configuration; no imbalance correction.	Classical baseline.
XGBoost - weighted	$\text{scale_pos_weight} = n_{\text{observed_majority}} / n_{\text{observed_minority}}$.	Common algorithm-level imbalance correction; trusts noisy observed class counts and upweights the observed minority, so its failure mode can differ by direction.
XGBoost - random oversampling	RandomOverSampler to balance observed classes, then frozen XGBoost.	Duplicates observed minority-labeled rows; can amplify false rare examples in Direction A and duplicate a reduced genuine minority pool in Direction B.
XGBoost - SMOTE	SMOTE on training fold only, then frozen XGBoost.	Synthetic minority correction; tests distortion from false rare examples in Direction A and from an incomplete minority manifold in Direction B.

7.1 Oracle-prior diagnostic (not a deployable method)

For thresholding, DistPFN, and DistPFN-T, rerun the post-hoc adjustment using the clean pre-contamination class prior π_{true} instead of the noisy observed prior π_{obs} . This is explicitly an oracle diagnostic. It tells us whether correction failure is largely caused by prior corruption. It must never be reported as a practical production method.

7.2 Frozen model configuration

Model	Frozen configuration	Notes
TabPFN	Use one fixed current TabPFN checkpoint for the entire study; no task-specific gradient training. Record package version, model/checkpoint identifier, device, and inference configuration.	Do not change checkpoint after inspecting results. GPU recommended.
XGBoost	n_estimators=500, max_depth=6, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, eval_metric="logloss", random_state=seed.	Use identical settings for no-correction, weighted, ROS, and SMOTE variants.
Optional CatBoost sanity check	iterations=500, depth=6, learning_rate=0.05, verbose=False, random_seed=seed.	Run only after core results; not needed for the central claim.
Optional ITBoost	Use authors' official implementation if installable quickly and reproducibly.	Useful label-noise-aware comparator, but do not risk the three-day schedule.

8. Preprocessing and Data Leakage Controls

1. Split first: create clean stratified train/test partitions before fitting any transformer.
2. Build the controlled clean training sample at target r from the training partition only.
3. Inject either directed label-error mechanism only after clean sampling; never alter test labels.
4. Fit numeric median imputation on the current training condition only. Standardize only if a method requires it; tree models and TabPFN generally do not need standard scaling.
5. Fit categorical imputation/one-hot encoding on current training data only; unknown test categories must be ignored rather than learned from test data.
6. Apply resampling (downsampling, random oversampling, SMOTE) after the train split and preprocessing fit, never before the train/test split.
7. Correction thresholds and priors must come from training labels only. The oracle-prior ablation may use the pre-contamination training prior, never test labels.
8. No test-set metric may be used for threshold or model selection.

9. Evaluation Metrics

Metric	Status	Why it matters
Balanced Accuracy	Primary	Equal emphasis on both classes; directly comparable with PFN imbalance work.
Worst-Class Accuracy / Recall	Primary	Captures whether one class collapses even when average performance appears acceptable.
AUPRC	Primary	Sensitive to positive/minority ranking; especially informative in imbalanced problems.
Macro-F1	Primary	Balances precision and recall across classes.
Minority precision / recall	Diagnostic	Shows whether a correction creates false positives or misses rare events.
AUROC	Secondary	Threshold-free discrimination; useful but can conceal practical rare-class failure.
Brier score	Reliability	Proper scoring rule for probability quality.
Log loss	Reliability	Strongly penalizes confident wrong predictions, relevant under corrupted training labels.

Metric	Status	Why it matters
ECE	Reliability	Measures calibration error; use fixed binning and document the bin count.

Primary paper ranking should not be based on raw accuracy. Report per-dataset changes and cross-dataset ranks/paired differences because AUPRC depends on test prevalence and datasets have different natural class ratios.

10. New Contribution Metrics: Gain, Harm, Break-Even, Prior Sensitivity

10.1 Correction Gain

For higher-is-better score S , define the gain of correction m relative to the matching uncorrected base model under the exact same dataset, seed, true imbalance r , directed label-error condition d , and noise level c :

$$CG_m(r,c,d) = S_corrected_m(r,c,d) - S_uncorrected(r,c,d)$$

For lower-is-better reliability losses such as Brier and log loss, reverse the sign so positive CG always means the correction helped.

10.2 Correction Harm Rate (CHR)

$$CHR_m(d) = \text{count}[CG_m(r,c,d) < -\epsilon] / \text{count}[\text{evaluated corrupted cells in direction } d]$$

Use $\epsilon = 0$ for the descriptive headline and optionally report a practical threshold ϵ (for example 0.01 absolute balanced-accuracy points) as a sensitivity analysis. Always state which definition is used.

10.3 Correction Break-Even Point

$$c^*_m(r,d) = \text{smallest direction-specific noise } c > 0 \text{ such that } CG_m(r,c,d) \leq 0$$

To avoid declaring break-even from one noisy seed, compute CG after averaging the three seeds for each dataset and report the distribution of c^* across datasets. If a method stays beneficial at $c=0.30$, record c^* as " >0.30 " rather than extrapolating.

10.4 Harm Magnitude / Correction Regret

$$\text{Regret_m}(r,c,d) = \max(0, S_uncorrected(r,c,d) - S_corrected_m(r,c,d))$$

CHR tells how often a correction hurts; regret tells how badly it hurts when it does.

10.5 Prior Sensitivity Gap

$$PSG_m(r,c,d) = S_oracle_prior_m(r,c,d) - S_observed_prior_m(r,c,d)$$

A positive PSG for thresholding/DistPFN suggests that distortion of the observed class prior contributes to failure. Compare PSG separately for Direction A (prior inflation) and Direction B (prior deflation). A small PSG with poor overall performance suggests that failure is not explained by prior distortion alone.

11. Statistical Analysis

- Report mean +/- standard deviation across three seeds for each dataset-method-r-c-direction cell; use direction=clean for the shared c=0 condition.
- Use paired dataset-stress cells when comparing correction methods; never treat individual seed runs as independent datasets.
- Use a Friedman omnibus test for multiple correction pipelines across paired blocks where the number of blocks is adequate [15].
- If the omnibus test is significant, use paired Wilcoxon signed-rank tests for selected planned contrasts with Holm correction.
- Report effect sizes/paired differences and confidence intervals where practical; do not present p-values without the magnitude of the difference.
- Analyze break-even descriptively per dataset and imbalance level; with five datasets, avoid sweeping universal claims.
- If a pipeline fails technically, record the failure and reason. Do not silently discard cells or substitute a different model after seeing results.

12. Required Figures and Tables

Output	Content	Purpose
Figure 1	Experimental pipeline	Methodology overview.
Figure 2	Correction-gain heatmaps: $r \times c$ for each correction, faceted by label-error direction	Shows where each method helps or hurts under false-minority contamination versus rare-label loss.
Figure 3	Direction-specific break-even curves / markers across imbalance levels	Main practical result: how much of each directed error each correction tolerates.
Figure 4	CHR and mean regret by correction method and direction	Summarizes frequency and severity of harmful correction.
Figure 5	Observed-prior vs oracle-prior performance, separated by direction	Contrasts prior inflation (A) with prior deflation (B) for threshold/DistPFN.
Figure 6	Reliability degradation: Brier/ECE vs noise level, faceted by direction	Tests whether probability quality fails before discrimination.
Table 1	Dataset characteristics and natural class counts	Reproducibility.
Table 2	Exact package/model/correction configurations	Reproducibility.
Table 3	Balanced clean and severe-stress performance summary	Compact main performance table.
Table 4	Correction break-even c^* and CHR by direction	Central contribution table.
Table 5	Planned pairwise statistical comparisons	Supports claims without a giant leaderboard.

Results-table rule

Do not put every run in the manuscript. Keep the full long-form CSV in the repository/supplement. The paper body should show aggregated results and the mechanism-focused gain/harm/break-even summaries.

13. Implementation Blueprint

13.1 Repository structure

```
ml-correction-backfire/
|-- README.md
|-- requirements.txt
|-- requirements-lock.txt
|-- configs/
|   |-- experiment.yaml
|-- src/
|   |-- data.py
|   |-- build_imbalance.py
|   |-- contaminate.py
|   |-- preprocess.py
|   |-- tabpfn_methods.py
|   |-- xgb_methods.py
|   |-- metrics.py
|   |-- run_core.py
|   |-- analyze.py
|   |-- plots.py
|-- tests/
|   |-- test_imbalance.py
|   |-- test_contamination.py
|   |-- test_no_leakage.py
|-- results/
|   |-- raw_runs.csv
|   |-- summary.csv
|   |-- failures.csv
|-- figures/
|-- tables/
|-- manuscript/
```

13.2 Environment

```
Python >= 3.10
numpy
pandas
scipy
scikit-learn
matplotlib
xgboost
tabpfn
imbalanced-learn
ucimlrepo
statsmodels
tqdm
pyyaml
# optional: catboost, official ITBoost code
```

At experiment freeze, save `pip freeze > requirements-lock.txt` and record OS, CPU, GPU, CUDA/PyTorch, TabPFN package version, exact TabPFN checkpoint/model identifier, and all random seeds.

13.3 Core configuration file

```

datasets: [267, 176, 522, 94, 17]
seeds: [13, 37, 73]
true_minority_ratio: [0.50, 0.20, 0.10, 0.05]
label_error:
  rates: [0.00, 0.05, 0.10, 0.20, 0.30]
  directions:
    - majority_to_minority
    - minority_to_majority
zero_rate_shared_clean: true
methods:
  - tabpfn_none
  - tabpfn_threshold_obs
  - tabpfn_downsample
  - distpfn_obs
  - distpfn_t_obs
  - xgb_none
  - xgb_weighted
  - xgb_ros
  - xgb_smote
oracle_diagnostics:
  - tabpfn_threshold_oracle
  - distpfn_oracle
  - distpfn_t_oracle

```

13.4 Directed label-error functions

```

def majority_to_minority_contamination(y_clean, c, rng):
    # c = fraction of observed minority-labeled set that is truly majority
    y = y_clean.copy()
    n_min = int((y_clean == 1).sum())
    k = int(round((c / (1.0 - c)) * n_min)) if c > 0 else 0
    src = np.flatnonzero(y_clean == 0)
    assert k <= len(src)
    flip_idx = rng.choice(src, size=k, replace=False)
    y[flip_idx] = 1
    realized = k / max(1, n_min + k)
    return y, flip_idx, realized

def minority_to_majority_loss(y_clean, c, rng):
    # c = fraction of true minority labels omitted
    y = y_clean.copy()
    n_min = int((y_clean == 1).sum())
    k = int(round(c * n_min)) if c > 0 else 0
    src = np.flatnonzero(y_clean == 1)
    flip_idx = rng.choice(src, size=k, replace=False)
    y[flip_idx] = 0
    realized = k / max(1, n_min)
    return y, flip_idx, realized

# c=0 is stored once as a shared clean condition;
# for c>0, run both directions from the same clean rows.

```

13.5 TabPFN correction functions

```
# p is TabPFN P(y=1) on clean test rows
pi_obs = y_train_noisy.mean()
pi_true = y_train_clean.mean()

# McDowell-style threshold correction
yhat_threshold_obs = (p > pi_obs).astype(int)
yhat_threshold_oracle = (p > pi_true).astype(int)

# DistPFN for 2 classes
p2 = np.column_stack([1-p, p])
prior_obs = np.array([1-pi_obs, pi_obs])
adj = (p2 ** 2) / np.clip(prior_obs, 1e-8, None)
p_dist = adj / adj.sum(axis=1, keepdims=True)

# DistPFN-T: reproduce the authors' formula from the paper/code;
# validate against the official implementation on a toy example before core runs.
```

13.6 XGBoost correction functions

```
ratio_obs = (y_train_noisy == 0).sum() / max(1, (y_train_noisy == 1).sum())

# weighted
XGBClassifier(..., scale_pos_weight=ratio_obs)

# random oversampling
X_ros, y_ros = RandomOverSampler(random_state=seed).fit_resample(X_train, y_train_noisy)

# SMOTE
X_sm, y_sm = SMOTE(random_state=seed, k_neighbors=k).fit_resample(X_train, y_train_noisy)
# k must be <= minority_count - 1; record automatic reductions for tiny cells.
```

13.7 Long-form results schema

Field group	Required columns
Identity	dataset_id, dataset_name, seed, run_timestamp, git_commit
Stress design	N_common, true_ratio, noise_direction, requested_contamination, realized_contamination, n_flipped, pi_true, pi_observed, flipped_index_hash
Method	model_family, correction_method, prior_mode, model_version, checkpoint_id
Performance	balanced_accuracy, worst_class_accuracy, auprc, macro_f1, minority_precision, minority_recall, auROC
Reliability	brier, log_loss, ece
Derived later	correction_gain, harm_indicator, regret, break_even_by_direction, prior_sensitivity_gap
Operations	fit_seconds, predict_seconds, status, error_message

13.8 Minimal unit tests before full run

- At $c=0$, both directed-noise functions return exactly the clean labels and $n_{\text{flipped}}=0$; the runner stores only one shared clean stress condition.
- For each requested c and direction, the realized direction-specific noise rate is within rounding tolerance.
- At every r , the clean controlled training set has exactly N_{common} rows.

- The same dataset/seed/r/c/direction condition produces identical noisy labels regardless of which correction method is requested.
- The test row indices and test labels never change across r/c/direction/method within a seed.
- Random oversampling and SMOTE are called only on the training matrix.
- Observed-prior and oracle-prior variants are both logged with explicit prior_mode values.
- DistPFN implementation matches the paper formula and a small official-code sanity check before experiments begin.

14. Three-Day Execution Schedule

Day	Primary objective	Required deliverables / stop condition
Day 1 - Aug. 12	Freeze protocol and make one dataset run end-to-end	Create repo/environment; load all 5 datasets; implement constant-size imbalance and both directed label-error generators; validate TabPFN; implement threshold/DistPFN/XGB corrections; complete unit tests; run one dataset x one seed across all 36 unique stress cells. Stop Day 1 only when raw_runs.csv is produced correctly.
Day 2 - Aug. 13	Complete evidence and freeze results	Run all core cells for both directions; rerun only documented technical failures; freeze raw CSV; compute direction-specific CG/CHR/regret/break-even/PSG; generate all main figures/tables; optionally run CatBoost/ITBoost only if the core is complete.
Day 3 - Aug. 14	Write the manuscript around verified results	Write Methods first; then Results from frozen outputs; Introduction/Related Work; Discussion; Threats; Abstract last; verify every number against table/CSV; package code/config/results and references.

14.1 End-to-End Step-by-Step Execution Protocol

Step 1 - Freeze the research question

Copy the final title, RQs, hypotheses, imbalance levels, both label-error directions, noise levels, seeds, core methods, and primary metrics into configs/experiment.yaml. Do this before looking at comparative results.

Step 2 - Create the repository

Create the folder structure in Section 13.1. Initialize version control if available. Add README.md, configs/, src/, tests/, results/raw/, results/summary/, figures/, and manuscript/.

Step 3 - Create a clean Python environment

Use Python 3.10 or newer. Install numpy, pandas, scipy, scikit-learn, matplotlib, xgboost, tabpfn, imbalanced-learn, ucimlrepo, statsmodels, tqdm, and pyyaml. Install optional packages only after the core pipeline works.

Step 4 - Freeze software identity

Record pip freeze, operating system, CPU/GPU, CUDA/PyTorch version, exact TabPFN package version, and the exact model/checkpoint identifier. The study must not switch model checkpoints after results are seen.

Step 5 - Download only the five audited UCI datasets

Fetch UCI IDs 267, 176, 522, 94, and 17. Save an immutable raw copy or reproducible fetch manifest; never manually edit raw files.

Step 6 - Audit every dataset

Log rows, predictor columns, dtypes, duplicate-row count, missing-value count per column, target values, target counts, and any identifier columns. Resolve target semantics from UCI documentation rather than guessing from numeric labels.

Step 7 - Define the positive/minority class

Choose one semantically fixed positive class for each dataset and encode it as 1. This mapping must stay unchanged even when controlled sampling makes the class balanced.

Step 8 - Create three stratified clean splits

For seeds 13, 37, and 73, create the clean train/test split before imbalance or either directed label error. Save train/test indices or hashes so the same split is reused by every method.

Step 9 - Freeze the clean test set

Do not resample, corrupt, rebalance, tune on, or fit preprocessing using test labels. Within a dataset/seed, test rows and labels must remain identical for every r , c , direction, and method.

Step 10 - Compute the common training size

For each dataset/seed, compute N_{common} so every target minority prevalence r uses the same total clean training size. This removes total-sample-size as a confound.

Step 11 - Build each clean controlled imbalance set

For r in $\{0.50, 0.20, 0.10, 0.05\}$, sample N_{common} rows from the clean training pool to meet the requested true minority prevalence. Save selected row indices.

Step 12 - Verify controlled imbalance

Assert exact total size, expected minority/majority counts within rounding tolerance, no overlap with the test set, and identical N_{common} across r .

Step 13 - Create both directed label-error draws per nonzero stress level

For $c=0$, keep one shared clean condition. For each c in $\{0.05, 0.10, 0.20, 0.30\}$, create two deterministic noisy-label copies from the same clean controlled training rows: Direction A flips majority labels to minority until c is the target contamination fraction of the observed minority-labeled set; Direction B flips $\text{round}(c \times N_{\text{min}})$ true minority labels to majority so c is the minority-label loss rate. Save flipped indices separately for each direction.

Step 14 - Verify both directed label-error mechanisms mathematically

At $c=0$ labels must be identical. For Direction A, verify the realized observed-minority contamination fraction and confirm $\pi_{\text{obs}} \geq \pi_{\text{true}}$. For Direction B, verify the realized minority-label loss rate and confirm $\pi_{\text{obs}} \leq \pi_{\text{true}}$. In both cases log requested c , realized c , n_{flipped} , π_{true} , π_{obs} , and unchanged feature hashes.

Step 15 - Reuse the same directed noisy labels across methods

All competing corrections for a dataset/seed/ r/c /direction cell must receive the identical X_{train} and $y_{\text{train_noisy}}$. Never redraw label errors separately for one model.

Step 16 - Fit preprocessing without leakage

Fit imputers/encoders on the current training condition only. Apply transformations to the fixed test data. Audit Spambase missingness rather than assuming metadata is correct.

Step 17 - Implement TabPFN baseline

Run the frozen TabPFN model on the noisy context and obtain class probabilities. Save raw probabilities because thresholding, DistPFN, and DistPFN-T are post-hoc transformations and should reuse the same base prediction where applicable.

Step 18 - Implement McDowell-style threshold correction

For binary minority class 1, classify as minority when $p(y=1) > \pi_{\text{obs}}$. Add a diagnostic oracle version using π_{true} . The oracle is not deployable and must be labeled as such.

Step 19 - Implement TabPFN downsampling

Downsample the observed majority-labeled training rows to balance the observed labels, then rerun TabPFN. Log how many rows remain so computation reduction is visible.

Step 20 - Implement DistPFN exactly from the paper

Apply $\text{Norm}[p_{\text{hat}}(y)^2 / p_{\text{train}}(y)]$ using the observed noisy training prior for the deployable stress test. Normalize over classes. Unit-test probabilities sum to one and compare a toy case with the official code.

Step 21 - Implement DistPFN-T exactly from the paper

Compute tau from cross-entropy between predicted distribution and training prior, form the temperature-scaled predicted distribution, then apply the paper's normalized adjustment. Validate against official code before full runs.

Step 22 - Implement observed-prior versus oracle-prior diagnostics

For thresholding, DistPFN, and DistPFN-T, keep the main observed-prior result and add a clearly marked oracle clean-prior diagnostic. The difference becomes Prior Sensitivity Gap.

Step 23 - Implement XGBoost baseline

Use one frozen configuration for all XGBoost variants. Do not tune separate hyperparameters by stress condition.

Step 24 - Implement weighted XGBoost

Set `scale_pos_weight` from observed noisy class counts. This deliberately tests what happens when loss weighting trusts contaminated labels.

Step 25 - Implement RandomOverSampler + XGBoost

Balance observed labels by duplicating minority-labeled rows only within the training set. Under Direction A this tests amplification of false rare labels; under Direction B it tests oversampling from a reduced set of correctly labeled minority examples.

Step 26 - Implement SMOTE + XGBoost

Apply SMOTE only to the training data after splitting. If too few observed minority points exist for the default neighbor count, choose a deterministic valid k based only on training counts and log it; otherwise mark the cell failed rather than silently changing the design.

Step 27 - Add optional robust baselines only after the core matrix exists

ITBoost or other robust-GBDT methods may be informative, but they are optional. Do not delay the central study or alter the core hypothesis because an optional repository is difficult to run.

Step 28 - Implement all metrics once

Compute balanced accuracy, worst-class recall/accuracy, AUPRC, macro-F1, minority precision/recall, AUROC, Brier score, log loss, and ECE using one tested metrics module.

Step 29 - Define score direction consistently

For higher-is-better metrics, correction gain is corrected minus baseline. For losses such as Brier/log loss, reverse the sign so positive gain always means correction helped.

Step 30 - Run unit tests

Run every test in Section 13.8. Add tests for DistPFN normalization, threshold=prior behavior, deterministic draws for both label-error directions, no train/test overlap, and stable split hashes.

Step 31 - Run a one-cell smoke test

Use one dataset, one seed, one r , one nonzero c , both directions, and all core methods. Also run the shared $c=0$ clean cell. Confirm all methods write valid `raw_run` rows and probabilities/metrics are finite.

Step 32 - Run a one-dataset pilot

Complete all 36 unique stress cells per seed for one dataset: 4 imbalance levels x [1 shared clean condition + 4 nonzero noise levels x 2 directions], across all three seeds. Inspect only for implementation errors, runtime, impossible metrics, or leakage - not to select a preferred method.

Step 33 - Freeze the protocol hash

Once the pilot is technically correct, save the config and code commit/hash. Any later design change must be documented with a reason before aggregate comparisons.

Step 34 - Run the full core matrix

Run 5 datasets x 3 seeds x 36 unique stress contexts per dataset/seed (4 imbalance levels x [1 shared clean + 4 nonzero rates x 2 directions]) across the nine core pipelines. This yields 540 unique stress contexts and 4,860 method-context evaluations before reuse of shared/post-hoc computation.

Step 35 - Record failures, do not erase them

Write failed cells to `failures.csv` with method, dataset, stress condition, exception/reason, and whether a deterministic retry succeeded.

Step 36 - Build the frozen summary table

Aggregate seeds within each dataset-method- r - c -direction cell, with `direction='clean'` for $c=0$. Preserve `raw_runs.csv` separately. Generate `summary.csv` only with a deterministic script.

Step 37 - Compute correction-centric quantities

Compute Correction Gain, Correction Harm Rate, Correction Regret, break-even noise c^* , and Prior Sensitivity Gap exactly as defined in Section 10, separately for each directed error mechanism before any cross-direction synthesis.

Step 38 - Perform planned statistical tests

Use paired blocks, Friedman for multi-method omnibus comparisons when appropriate, then planned paired Wilcoxon tests with Holm correction. Report effect magnitudes, not p-values alone.

Step 39 - Generate all manuscript figures from CSV

Create direction-faceted gain heatmaps, direction-specific break-even curves, CHR/regret summaries, observed-vs-oracle plots, and reliability curves from the frozen data files. Never hand-enter numbers.

Step 40 - Interpret without forcing the hypothesis

If a correction never becomes harmful, report that. If a method fails only on certain datasets, report heterogeneity. The paper is valuable as a robustness characterization even if the expected backfire effect is weak.

Step 41 - Write Methods before Results narrative

Use the pre-registered design, exact formulas, software versions, dataset audit, leakage controls, and run counts. Methods should be reproducible without looking at the result tables.

Step 42 - Write Results directly from frozen outputs

Answer RQ1-RQ5 in order, keeping Direction A and Direction B results separate unless an explicitly defined aggregate is reported. Every numerical statement must be traceable to summary.csv/raw_runs.csv and figure-generation code.

Step 43 - Write Discussion with bounded claims

Explain mechanisms as interpretations, distinguish false-minority amplification from true-minority omission and prior distortion, compare findings to McDowell/DistPFN/noise literature, and list counterexamples.

Step 44 - Refresh the literature immediately before submission

Repeat the post-CLIMB/2026 search. If a directly overlapping paper appears, revise the novelty statement and positioning rather than ignoring it.

Step 45 - Final reproducibility and integrity audit

Re-run sampled cells, rebuild figures from raw results, verify citations/venue status, remove placeholders, confirm no hypothetical values remain, and ensure the repository can regenerate every paper table/figure.

14.2 Runtime Fallback Rule

If compute becomes the bottleneck, do not cherry-pick datasets based on performance. Reduce optional methods first (CatBoost/ITBoost), then oracle diagnostics if necessary. Both directed label-error mechanisms remain part of the core design. The nine core pipelines, five datasets, three seeds, and 36 unique stress contexts per dataset/seed are the planned target. Any reduction must be documented before comparing winners.

15. Paper Structure

Section	Target length	Content
Abstract	180-220 words	Problem, controlled design, actual headline result, break-even/harm insight, practical implication. Write last.
1. Introduction	700-900	Why rare classes matter; why correction can amplify bad supervision; post-CLIMB context; gap; RQs; contributions.
2. Related Work	800-1,000	Imbalanced tabular learning (CLIMB/TILBench); PFN imbalance correction; label-noise robustness; DistPFN/label shift; reliability.
3. Methodology	1,200-1,500	Datasets; split; constant-size imbalance; two directed label-error mechanisms and formulas; correction methods; oracle diagnostic; metrics; statistics.
4. Results	1,000-1,400	RQ1-RQ5 in order, reporting Direction A and Direction B separately. Lead with correction gain and break-even, not a raw leaderboard.

Section	Target length	Content
5. Discussion	700-1,000	Why corrections fail/survive; false-minority amplification vs true-minority omission; prior inflation vs deflation; implications for practitioners and TFM design.
6. Threats to Validity	350-500	Synthetic directed label errors, 5 datasets, binary only, fixed models, version dependence, oracle diagnostic.
7. Conclusion	250-350	What was learned and the practical rule: correction is conditional on label trustworthiness.

15.1 Intended contribution paragraph

This study makes four empirical contributions: (1) a controlled bidirectional protocol that separately models majority-to-minority contamination and minority-to-majority label loss while independently controlling true minority prevalence; (2) a direct stress test of recent TabPFN imbalance/label-shift corrections and established GBDT imbalance remedies under both mechanisms; (3) direction-specific correction-centric metrics - gain, harm rate, regret, and break-even noise - that quantify when mitigation itself becomes harmful; and (4) an oracle-prior diagnostic that isolates the role of inflated versus deflated class-prior information in prior-dependent correction failure.

16. Threats to Validity

Threat	Why it matters	Mitigation / required wording
Only five datasets	Cannot establish universal ranking across tabular ML.	Use cross-domain datasets, paired analysis, and explicitly frame results as controlled evidence.
Synthetic contamination	Real annotation mistakes may be feature-dependent or systematic.	Frame both directions as reproducible stress tests; majority->minority and minority->majority still do not capture feature-dependent or real annotator behavior; recommend future real-noise validation.
Binary classification only	Correction behavior may differ in multiclass settings.	State binary scope explicitly; multiclass is future work.
Artificial class ratios	Controlled r may differ from naturally long-tailed data.	Retain natural-reference condition and constant-size design.
Model/checkpoint drift	TabPFN software evolves quickly.	Freeze exact version/checkpoint and log it in every run.
No exhaustive hyperparameter tuning	May understate tuned classical baselines.	Use frozen fair configurations; paper is about correction robustness, not absolute SOTA.
Oracle prior	Uses information unavailable in deployment.	Label it diagnostic only; never present it as a practical method.
SMOTE assumptions	Synthetic interpolation may be awkward after one-hot encoding or with tiny minority cells.	Use training-only SMOTE, record $k_neighbors$, and skip with documented reason if mathematically infeasible.

17. Reproducibility and Run-Logging Rules

- Every row in the final tables must trace to a raw_run row and a deterministic analysis script; every noisy run must also identify its label-error direction.
- Freeze the experiment YAML before the full matrix begins; save a copy with the results.
- Log exact dataset ID, row indices or split hash, seed, label-error direction, flipped-label indices, model version, and correction configuration.
- Generate publication figures directly from the frozen summary CSV; do not manually type plot values.
- Preserve failures.csv with stack trace or reason for every unsuccessful cell.
- Never remove an outlier run merely because it changes the narrative. Investigate only reproducibility/implementation failures.
- Include a README command that rebuilds summaries and figures from raw_runs.csv without retraining models.
- Before submission, rerun a random sample of cells from scratch to verify deterministic regeneration within expected numerical tolerance.

18. Decision Logic After Results

Use the evidence to choose the narrative. Do not decide the winner in advance.

Observed result	Appropriate interpretation
Threshold/DistPFN lose benefit in one or both directions but oracle-prior versions recover	Correction failure is partly driven by distortion of the observed class prior; compare prior inflation (A) with prior deflation (B).
ROS/SMOTE show high CHR under majority->minority contamination	Minority amplification can amplify false minority labels; mitigation should be conditional on trust in the observed minority-labeled pool.
Minority->majority label loss causes larger recall/prior failures than majority->minority contamination	Losing genuine rare examples can be a distinct failure mechanism from adding false rare examples; report the asymmetry rather than pooling the directions.
All corrections remain robust through $c=0.30$ in both directions	Important negative result: these corrections tolerate substantial directed rare-label error under this controlled protocol.
AUROC stays high while Brier/ECE worsens	Ranking performance hides probability-reliability degradation; deployment confidence requires calibration checks.
Results are strongly dataset-dependent	Emphasize regime dependence and avoid a universal method recommendation.
Uncorrected baseline often wins under one or both directed error mechanisms	The practical finding is that correction can backfire when label quality is poor, and the failure threshold may depend on error direction.

19. References

- [1] Z. Liu, Z. Li, Z. Yang, T. Wei, J. Kang, Y. Zhu, H. Hamann, J. He, and H. Tong, "CLIMB: Class-imbalanced Learning Benchmark on Tabular Data," Advances in Neural Information Processing Systems 38, Datasets and Benchmarks Track, 2025. arXiv:2505.17451.
- [2] R. Liu and J. Luo, "TILBench: A Systematic Benchmark for Tabular Imbalanced Learning Across Data Regimes," arXiv:2605.14915, May 2026.
- [3] S. McDowell, N. Stromberg, and L. Sankar, "Correcting Class Imbalance in Prior-Data Fitted Networks for Tabular Classification," arXiv:2605.21742v1, May 20, 2026.
- [4] S. Lee, J. Lee, J. Seo, S. Yoo, M. Kim, T. Y. Lim, D. Kang, H. Choi, S. Lee, and W. Ahn, "Mitigating Label Shift in Tabular In-Context Learning via Test-Time Posterior Adjustment," ICML 2026; arXiv:2605.04363v2, May 24, 2026.

- [5] J. Hu and M. Ghelichi, "Noise Immunity in In-Context Tabular Learning: An Empirical Robustness Analysis of TabPFN's Attention Mechanisms," arXiv:2604.04868, 2026.
- [6] Y. Su, L. Zhao, D. Garcia-Gil, J. Guo, G. Zhang, J. Chen, and J. Chen, "ITBoost: Information-Theoretic Trust for Robust Boosting," arXiv:2605.04671, 2026.
- [7] J. L. De Melo Costa, F. Popineau, A. Rimmel, and B.-L. Doan, "High Performance, Low Reliability: Uncertainty Benchmarking for Tabular Foundation Models," Proceedings of ESANN 2026, 2026. doi:10.14428/esann/2026.ES2026-261; arXiv:2605.28554.
- [8] Y. He, Z. Guan, A. Wu, P. Cui, F. Wu, and K. Kuang, "SkillTFM: Gated Skill Evolution for Training-Free Adaptation of Tabular Foundation Models," arXiv:2608.06137, Aug. 6, 2026.
- [9] N. Hollmann, S. Muller, L. Purucker, A. Krishnakumar, M. Korfer, S. B. Hoo, R. T. Schirrmeyer, et al., "Accurate predictions on small data with a tabular foundation model," Nature, vol. 637, pp. 319-326, 2025. doi:10.1038/s41586-024-08328-6.
- [10] J. Qu, D. Holzmüller, G. Varoquaux, and M. Le Morvan, "TabICL: A Tabular Foundation Model for In-Context Learning on Large Data," Proceedings of the 42nd International Conference on Machine Learning, PMLR 267, pp. 50817-50847, 2025.
- [11] B. Frenay and M. Verleysen, "Classification in the Presence of Label Noise: A Survey," IEEE Transactions on Neural Networks and Learning Systems, vol. 25, no. 5, pp. 845-869, 2014.
- [12] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321-357, 2002.
- [13] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785-794, 2016. doi:10.1145/2939672.2939785.
- [14] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," PLOS ONE, vol. 10, no. 3, e0118432, 2015.
- [15] J. Demsar, "Statistical Comparisons of Classifiers over Multiple Data Sets," Journal of Machine Learning Research, vol. 7, pp. 1-30, 2006.
- [16] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," Proceedings of the 34th International Conference on Machine Learning, PMLR 70, pp. 1321-1330, 2017.
- [17] V. Lohweg, "Banknote Authentication" [Dataset], UCI Machine Learning Repository, 2012. doi:10.24432/C55P57.
- [18] I.-C. Yeh, "Blood Transfusion Service Center" [Dataset], UCI Machine Learning Repository, 2008. doi:10.24432/C5GS39.
- [19] "South German Credit" [Dataset], UCI Machine Learning Repository, 2019. doi:10.24432/C5X89F. UCI ID 522.
- [20] M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, "Spambase" [Dataset], UCI Machine Learning Repository, 1999. doi:10.24432/C53G6X.
- [21] W. Wolberg, O. Mangasarian, N. Street, and W. Street, "Breast Cancer Wisconsin (Diagnostic)" [Dataset], UCI Machine Learning Repository, 1993. doi:10.24432/C5DW2B.
- [22] Z. Ye, J. Kim, J. Sundberg, B. Varici, and P. Ravikumar, "Tabular Numeric Stretch Transformation," arXiv:2608.09162v1, Aug. 10, 2026.
- [23] J. Luo, Y. Quan, and S. Xu, "Robust-GBDT: GBDT with Nonconvex Loss for Tabular Classification in the Presence of Label Noise and Class Imbalance," arXiv:2310.05067, 2023 (revised 2024).
- [24] A. Eisenbürger, D. Otten, A. Hudde, and F. Hopfgartner, "Training Gradient Boosted Decision Trees on Tabular Data Containing Label Noise for Classification Tasks," arXiv:2409.08647v2, Jan. 6, 2025.
- [25] J. Zhang, M. Li, Q. Zhang, and Y. Zhu, "Active Learning with Foundation Model Priors: Efficient Learning under Class Imbalance," arXiv:2606.07630, May 30, 2026.

19.1 Primary online sources to re-check before submission

- Tabular Numeric Stretch Transformation (arXiv:2608.09162)
- Robust-GBDT (arXiv:2310.05067)
- South German Credit UCI 522 metadata / correction note
- [CLIMB arXiv](#)
- [TILBench arXiv](#)
- [PFN imbalance correction](#)
- [DistPFN](#)
- [Noise Immunity](#)
- [ITBoost](#)
- [Uncertainty benchmark](#)
- [SkillTFM](#)

Appendix A. Immediate Start Checklist

- ☐ Create repository and virtual environment; freeze config file before full experiments.
- ☐ Install TabPFN and verify predict_proba on a toy binary dataset; record model/checkpoint identifier.
- ☐ Download all five UCI datasets and save natural class counts/target mappings.
- ☐ Implement `compute_common_training_size()` and assert identical N_{common} across r values.
- ☐ Implement both `majority_to_minority_contamination()` and `minority_to_majority_loss()`; unit-test each realized direction-specific rate and prior shift.
- ☐ Implement TabPFN threshold correction using $\tau_{\text{obs}} = \pi_{\text{obs}}$ and the oracle τ_{true} diagnostic.
- ☐ Implement DistPFN formula and validate it against the authors' algorithm/official implementation on a toy example.
- ☐ Implement XGBoost baseline, weighting, RandomOverSampler, and SMOTE pipelines.
- ☐ Implement balanced accuracy, WCA, AUPRC, macro-F1, precision/recall, AUROC, Brier, log loss, ECE.
- ☐ Run one dataset x one seed x all 36 unique stress cells x all core methods; inspect direction labels, logs, and raw CSV.
- ☐ Launch the full core matrix only after leakage assertions and correction sanity checks pass.
- ☐ Do not start optional CatBoost or ITBoost experiments until the full bidirectional core raw CSV exists.

Appendix B. Manuscript Finalization Checklist

- ☐ Title names correction robustness/minority-label contamination rather than making a generic imbalance+noise claim.
- ☐ Introduction cites CLIMB, TILBench, PFN imbalance correction, DistPFN, Noise Immunity, ITBoost, SkillTFM, and uncertainty work as appropriate.
- ☐ Methods define r , c , and direction d mathematically and give separate formulas for majority->minority contamination and minority->majority label loss.
- ☐ Methods state that the test set is clean and fixed for each dataset/seed.
- ☐ Methods state the constant N_{common} rule and why it removes training-size confounding.
- ☐ The oracle-prior ablation is clearly labeled non-deployable/diagnostic.
- ☐ Every results claim is backed by the frozen CSV; no placeholder or hypothetical numbers remain.
- ☐ Correction gain, harm rate, regret, break-even, and prior-sensitivity formulas use consistent sign conventions.
- ☐ Figures show dataset aggregation and uncertainty/seed handling in captions.
- ☐ Statistical tests use paired blocks; multiple comparisons are corrected.
- ☐ Discussion distinguishes "correction hurts" from "base model fails" and avoids causal claims not supported by the protocol.
- ☐ Threats include synthetic directed label errors, binary scope, limited datasets, frozen tuning, model version dependence, and SMOTE assumptions.
- ☐ References are re-checked for any revisions or new papers published after August 12, 2026.
- ☐ Code/config/results package can regenerate every table and figure.

Appendix C. Source Verification Note

The literature framing in this blueprint was re-checked against primary/official sources through August 12, 2026. The final update explicitly accounts for CLIMB (NeurIPS 2025), TILBench (May 2026), PFN imbalance correction (May 20, 2026), DistPFN/DistPFN-T (ICML 2026; arXiv v2 May 24), TabPFN noise-immunity work (April 2026), ITBoost (May 2026), ESANN uncertainty benchmarking, SkillTFM (August 6, 2026), and Tabular Numeric Stretch Transformation (August 10, 2026). It also adds Robust-GBDT and a GBDT label-noise study to prevent the false claim that imbalance-plus-noise is itself new. The corrected South German Credit dataset (UCI 522) replaces only

the flawed legacy Statlog German Credit representation inside the existing five-dataset plan; the overall design remains cumulative. Because this is a fast-moving area, repeat the literature search immediately before submission.

Appendix D. Final No-Hallucination and Evidence-Control Rules

- No result value may enter the abstract, results, discussion, conclusion, figure, or table until it exists in the frozen results files.
- No venue is stated unless verified from a proceedings/OpenReview/official source. Otherwise cite the work as an arXiv preprint.
- No method equation is paraphrased from memory. DistPFN/DistPFN-T and PFN thresholding must be implemented from the paper equations and checked against official code where available.
- No dataset property is assumed from a secondary blog. Use official UCI metadata and programmatic audit; discrepancies are logged rather than hidden.
- No absolute novelty claim is permitted from a finite search. Use the narrower "we did not identify... as of the cutoff" language and refresh the search before submission.
- No cherry-picking. Dataset/method removal must be triggered by pre-defined technical constraints and recorded before performance comparison.
- No test-set tuning. The clean test set is only for final evaluation within each fixed experimental cell.
- No post-hoc model switching. Exact TabPFN/XGBoost versions and configurations are frozen after the technical pilot.
- No silent failure deletion. failures.csv is part of the reproducibility package.
- No manual result transcription when an analysis script can generate the value.

Appendix E. Verified Literature Status as of August 12, 2026

Work	Verified status	Primary evidence used	Role in this project
CLIMB	NeurIPS 2025 Datasets & Benchmarks	Official NeurIPS proceedings; 73 datasets, 29 CIL algorithms	Benchmark background; establishes data-quality importance.
TILBench	arXiv preprint, May 14 2026	arXiv:2605.14915	Shows generic imbalance benchmarking is already crowded: >40 algorithms, 57 datasets, >200k experiments.
Correcting Class Imbalance in PFNs	arXiv preprint, May 20 2026	arXiv:2605.21742v1	Direct source for threshold $\tau = \pi_{\text{minority}}$ and downsampling behavior; no unverified venue claim retained.
DistPFN / DistPFN-T	ICML 2026; arXiv v2 May 24 2026	ICML OpenReview + arXiv:2605.04363v2	Direct source for posterior-adjustment formulas and prior dependence.
Noise Immunity in In-Context Tabular Learning	arXiv preprint, Apr. 6 2026	arXiv:2604.04868	Shows base TabPFN robustness to controlled label noise; does not test our correction-failure mechanism.
ITBoost	arXiv preprint, May 6 2026	arXiv:2605.04671	Recent noise-robust boosting context; optional baseline.
High Performance, Low Reliability	ESANN 2026 proceedings	ESANN proceedings + arXiv:2605.28554	Motivates reliability/calibration analysis beyond discrimination.
SkillTFM	arXiv preprint, Aug. 6 2026	arXiv:2608.06137v1	Very recent broader TFM adaptation work; included to avoid stale positioning.
Tabular Numeric Stretch Transformation	arXiv preprint, Aug. 10 2026	arXiv:2608.09162v1	Newest directly adjacent tabular-noise robustness source found; Appendix G keeps validation/test clean.

Work	Verified status	Primary evidence used	Role in this project
Robust-GBDT	arXiv preprint	arXiv:2310.05067	Explicit evidence that noisy + imbalanced tabular learning is not a novel general problem.
Training GBDTs with Label Noise	arXiv preprint v2, Jan. 6 2025	arXiv:2409.08647v2	Classical tabular label-noise robustness context.

Literature-search limitation: this status table documents sources actually verified for the blueprint. It does not prove that no unpublished, unindexed, or newly posted competing work exists. Perform the final refresh immediately before submission.